IJM

IJM Disposal of Items

Setup & deployment guide — Firebase + Flutter web app (macOS)

Project ID: ijm-disposal · Code: github.com/emmanukw/ijm_disposal

What this guide covers

This guide takes you from nothing to a live booking website at **https://ijm-disposal.web.app**. Do the steps in order. Steps 1–4 happen in the Firebase website (console); steps 5–8 happen in the **Terminal** app on your Mac.

Step	What you do	Where
1	Create the Firebase project	Firebase console
2	Create the Firestore database	Firebase console
3	Turn on sign-in methods (Anonymous + Email/Password)	Firebase console
4	Create your admin account	Firebase console
5	Install the tools (Homebrew, Git, Node.js, Flutter, Chrome)	Mac Terminal
6	Download the code and connect it to Firebase	Mac Terminal
7	Upload security rules and test on your Mac	Mac Terminal + Chrome
8	Put the site online (Firebase Hosting)	Mac Terminal

How to run a Terminal command

Open **Terminal** (press Cmd+Space, type “Terminal”, press Enter). Copy one command from a dark box, paste it, press **Enter**, and wait until it finishes (you see the prompt again) before the next one.

STEP 1 OF 8

Create the Firebase project

Firebase hosts the website and stores the data.

1. Open **https://console.firebase.google.com** and sign in with the Google account IJM should own this under (an organisation account is better than a personal one).
2. Click **Create a project** (or **Add project**).
3. **Project name:** `ijm-disposal`. Note the **Project ID** shown below the name — in this setup it is `ijm-disposal`.
4. **Google Analytics:** turn it **off** (not needed).

5. Click **Create project**, wait, then **Continue**.
6. Stay on the free **Spark** plan. Say no if it suggests upgrading to Blaze.

STEP 2 OF 8

Create the Firestore database

This is where items, bids and settings are stored.

1. Left menu: **Build → Firestore Database** (may also be under **Databases** or **All products**).
2. Click **Create database**.
3. If asked for an edition, choose **Standard**.
4. **Database ID:** leave as `(default)` — the app expects this name.
5. **Location:** for Kenya choose **europe-west1 (Belgium)** or another Europe region. This cannot be changed later.
6. **Security rules:** choose **Start in production mode** (everything locked until Step 7).
7. Click **Create** and wait for the empty data screen. Don't add collections yet.

STEP 3 OF 8

Turn on sign-in methods

Bidders are signed in silently in the background; admins use email and password.

1. Left menu: **Build → Authentication → Get started**.
2. Open the **Sign-in method** tab.
3. Click **Anonymous** → switch on **Enable** → **Save**.
4. Click **Add new provider → Email/Password** → switch on the first **Enable** only (leave "Email link (passwordless)" off) → **Save**.
5. Check that both **Anonymous** and **Email/Password** show **Enabled**.

STEP 4 OF 8

Create your admin account

This is the account used to sign in at the website's /admin page.

Part A — Create the user

1. In **Authentication**, open the **Users** tab → **Add user**.
2. Enter the admin **email** and a strong **password** (keep it safe) → **Add user**.
3. Copy the user's **User UID** (long code in the last column).

Part B — Mark that user as an admin

1. Go to **Build → Firestore Database → + Start collection**.
2. **Collection ID:** `admins` (lowercase) → **Next**.
3. **Document ID:** paste the **User UID** (do not use Auto-ID).
4. Add one field: **Field** `email`, **Type** string, **Value** the admin's email.
5. Click **Save**.

Adding more admins later

Repeat Part A and Part B for each person: a new user in Authentication, plus a new document in `admins` whose ID is their UID.

STEP 5 OF 8

Install the tools on your Mac

You need Homebrew (app installer), Git, Node.js, Flutter and Google Chrome. Run each command in Terminal.

1. Install Homebrew (skip if `brew --version` already prints a version)

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

It asks for your Mac password (the cursor doesn't move while typing — that's normal). When it finishes, it prints **"Next steps"** with two or three commands: copy and run those too.

2. Install Git and Node.js

```
brew install git node
```

3. Install Flutter (large download — several minutes)

```
brew install --cask flutter
```

4. Install Google Chrome (if you don't have it)

```
brew install --cask google-chrome
```

5. Check everything

```
git --version && node --version && flutter --version
```

```
flutter doctor
```

In `flutter doctor`, only **Flutter** and **Chrome** need a green ✓. Warnings about Android, Xcode or iOS can be ignored — we only build for the web.

STEP 6 OF 8

Download the code and connect it to Firebase

Run these in Terminal, one at a time.

1. Download the code from GitHub

```
cd ~ && git clone https://github.com/emmanukw/ijm_disposal.git
```

```
cd ~/ijm_disposal
```

2. Install the Firebase command-line tools

```
npm install -g firebase-tools
```

If you get “permission denied” (EACCES), run it again with sudo in front.

3. Install the FlutterFire tool

```
dart pub global activate flutterfire_cli
```

```
echo 'export PATH="$PATH:$HOME/.pub-cache/bin"' >> ~/.zshrc && source ~/.zshrc
```

4. Sign in to Firebase

```
firebase login
```

A browser opens — choose the Google account that owns the Firebase project (from Step 1).

> **Switching to a different Google account**
> If it says you’re already logged in with the wrong account, run `firebase logout` then `firebase login` again (click “Use another account” in the browser). Check with `firebase projects:list` — **ijm-disposal** must be listed.
> **Important:** this must be the Google account that owns the project. The admin email from Step 4 exists only inside the app and cannot be used for `firebase login`.

Login	Which email	Used for
`firebase login` (Terminal)	Google account that created the Firebase project	Deploying and configuring from your Mac
Website admin page (/admin)	Admin email added in Step 4	Managing items, schedule, bids, reports

5. Connect the app to your project

```
flutterfire configure --project=ijm-disposal --platforms=web
```

If asked which web app to use, choose to **create a new web app** and accept the suggested name. It writes your real keys into `lib/firebase_options.dart`.

6. Link the deploy settings to your project

```
firebase use ijm-disposal
```

7. Download the app’s packages

```
flutter pub get
```

> **“15 packages have newer versions…” — is that a problem?**
> No. **“Got dependencies!”** means it worked. The project deliberately uses the versions it was built and tested with. Don’t run any upgrade command.

Check that configure worked (should show `projectId: 'ijm-disposal'`, not REPLACE_ME):

```
grep projectId lib/firebase_options.dart
```

STEP 7 OF 8

Upload security rules and test on your Mac

Upload the rules, then try the app as admin and bidder.

1. Upload the security rules

```
firebase deploy --only firestore:rules
```

It should end with **“Deploy complete!”**.

2. Run the app on your Mac

```
flutter run -d chrome
```

Chrome opens the app. It will say “Booking schedule not set yet” with no items — that’s expected.

3. Test as admin

1. Click **Admin** (top right) and sign in with the Step 4 email and password.
2. **Schedule:** set a closing date/time (e.g. tomorrow) → **Save schedule**.
3. **Items: Add item** — a photo, quantity 2, and a price.
4. Click **Public site**: you should see the countdown and the item card.

4. Test as a bidder

1. Click **Book now**, enter a name → **Confirm booking**.
2. The card should show 1 of 2 available, and the booking appears under **Live bids**.
3. In **Admin → Reports**, download the **Excel** file and open it.

Press **q** in Terminal to stop the app.

Is this a local deployment?

Partly. `flutter run` runs the website **only on your Mac** (a localhost address only you can see). `firebase deploy --only firestore:rules` is **online** — it updates your real project.

The **data is real**: even the local website reads and writes your live database. After testing, delete the test item (Admin → Items) and test bid (Admin → Bids).

STEP 8 OF 8

Put the site online

Publish the website to Firebase Hosting so others can use it.

1. Check the deploy target

```
firebase use
```

It must print **ijm-disposal**. If not, run `firebase use ijm-disposal`.

2. Build the production website

```
flutter build web --no-web-resources-cdn
```

Ends with **“✓ Built build/web”**. The option bundles everything into your own site so it loads reliably.

3. Publish it

```
firebase deploy --only hosting
```

Ends with **“Deploy complete!”** and the Hosting URL **https://ijm-disposal.web.app**.

4. Check from another device

Open the link on your phone using **mobile data** (not office Wi-Fi) to see what an outside bidder sees, then open `/admin` and sign in.

How others access the site

Once Step 8 is done, the website is online 24/7 — your Mac doesn't need to be on.

Who	Address	What they do
Bidders	https://ijm-disposal.web.app	See items and countdown, book, view live bids
Admins	https://ijm-disposal.web.app/admin	Sign in with the Step 4 email and password

- **Share the link** by email, WhatsApp, Teams, or a QR code on a notice board. No install and no account needed — bidders just type their name when booking.
- Alternative address (same site): https://ijm-disposal.firebaseapp.com
- **Cost:** free at this scale (Spark plan covers hundreds of bidders and thousands of bookings a day).
- **Anyone with the link can open it.** If it should be IJM staff only, a restriction can be added later (e.g. requiring an @ijm.org email or an access code).
- **Custom address (optional):** e.g. disposal.ijm.org — IT adds a DNS record; Firebase Hosting → Add custom domain.

Before sharing the link — checklist

- ☐ Delete the test bid (**Admin → Bids**) and test item (**Admin → Items**).
- ☐ Set the real title, opening and closing times in **Admin → Schedule**.
- ☐ Add the real items with photos, quantities and prices.
- ☐ Open the public link on a phone and confirm the countdown and items appear.

Publishing updates later

After any change to the app, run this from the `ijm_disposal` folder:

```
cd ~/ijm_disposal && flutter build web --no-web-resources-cdn && firebase deploy
--only hosting
```

During and after the event

- **Live bids** tab: everyone sees who booked what and when.
- **Admin → Bids:** cancel a booking — its pieces go back into stock.
- **Admin → Reports:** download **Excel** (sheets: Bids + Items Summary) or **CSV**. Times are shown in the admin's computer time zone.
- Booking stops automatically at the closing time (enforced by the server, not just the page).

Troubleshooting

Problem	Fix
`command not found: brew`	Run the "Next steps" commands Homebrew printed after installing, or open a new Terminal window.
`command not found: flutterfire`	Run the `echo 'export PATH=…'` command from Step 6.3, then open a new Terminal.

Problem	Fix
npm “permission denied” (EACCES)	Put `sudo` in front of the command.
`firebase projects:list` doesn’t show ijm-disposal	Wrong Google account: `firebase logout` then `firebase login`.
Admin sign-in says “Incorrect email or password”	Check the user exists in Authentication → Users, and Email/Password is enabled (Step 3).
Signed in but “not an admin”	The `admins` document ID must exactly match the user’s UID (Step 4 Part B). The screen shows the UID to use.
“Booking has closed” / Book buttons disabled	Set or extend the closing time in Admin → Schedule.
Booking fails with a permission error	Re-run `firebase deploy --only firestore:rules` and check a closing time is set.
Site shows old version after an update	Rebuild and redeploy (see “Publishing updates”), then refresh the page (Cmd+Shift+R).

Code repository: https://github.com/emmanukw/ijm_disposal · Firebase project: ijm-disposal